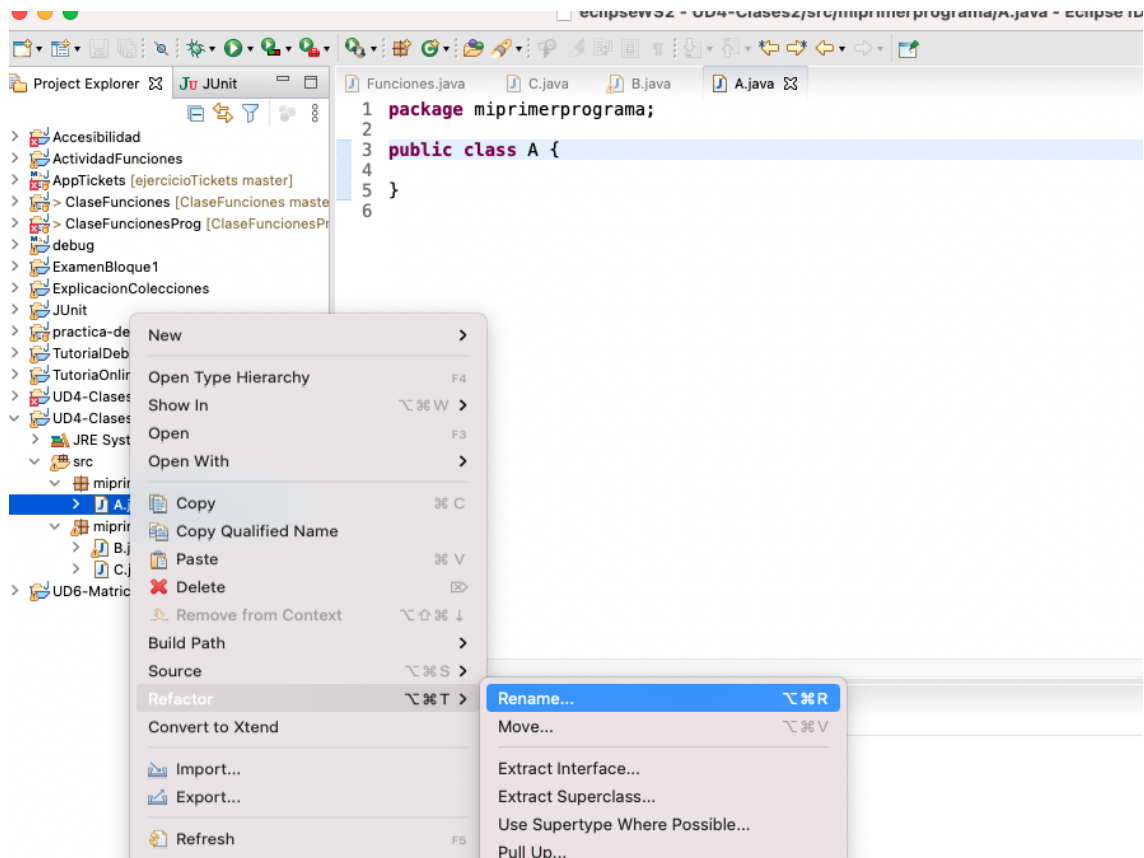


Renombrar

Es la opción más común, modifica el nombre a cualquier elemento (variable, atributo, método, clase...) y hace los cambios necesarios en las referencias que haya a dicho elemento en todo el proyecto.

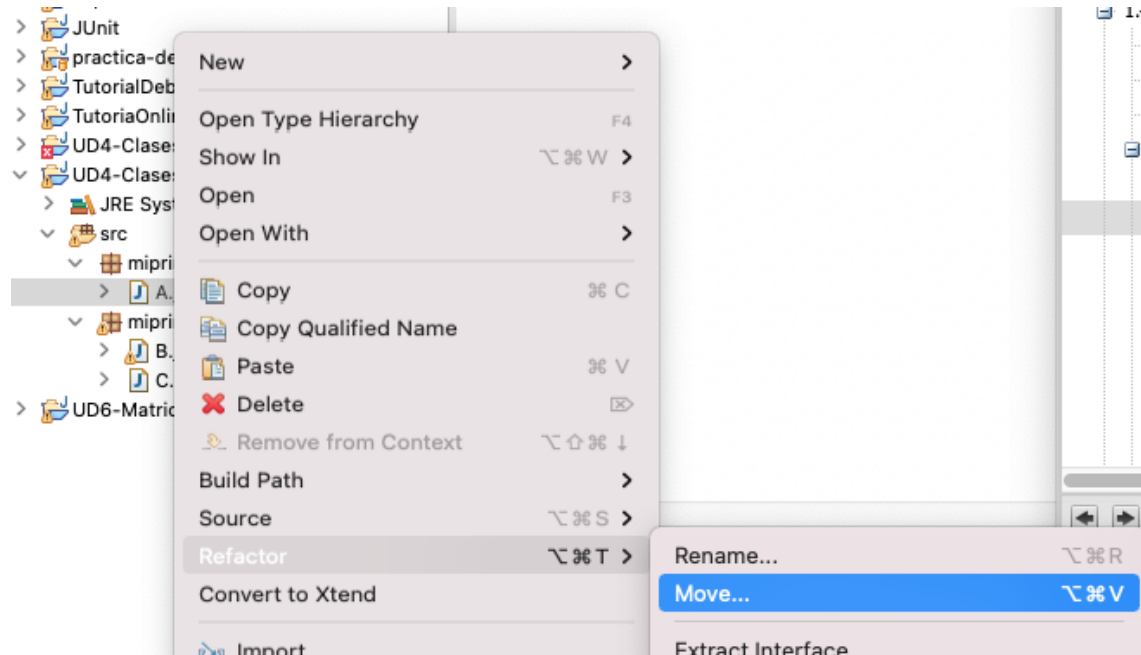
En la siguiente imagen se muestra como es posible modificar el nombre de la clase A de forma segura.



Mover

Cambia una clase de un paquete a otro, afectando a su declaración “package” y a su localización en el disco.

En la siguiente imagen se ve como es posible cambiar la localización de la clase A utilizando *move*.

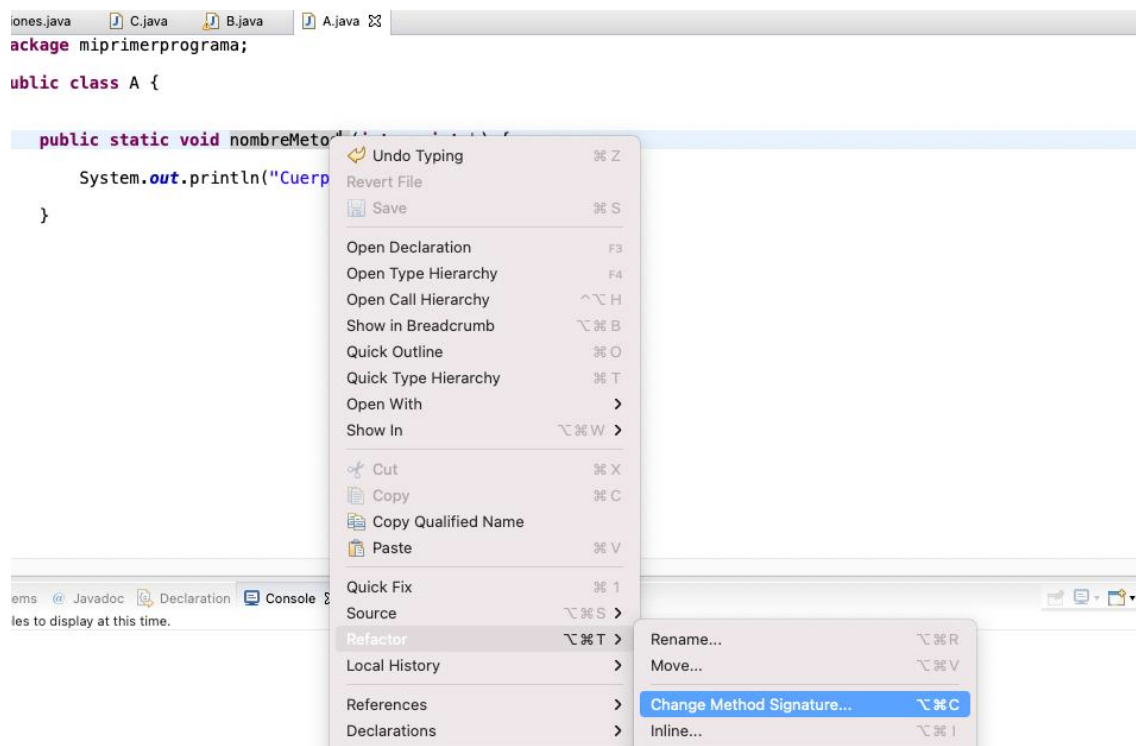


Cambiar signatura del método

Modifica la “firma” o cabecera del método. Si el método ha sido ya usado, cambiar el número o tipo de parámetros (así como el tipo de valor devuelto) provocará fallos de compilación.

Es útil para cambiar el nombre de los parámetros, o su orden (**Eclipse** modificará también el orden de entrada de los parámetros en todas las llamadas al método).

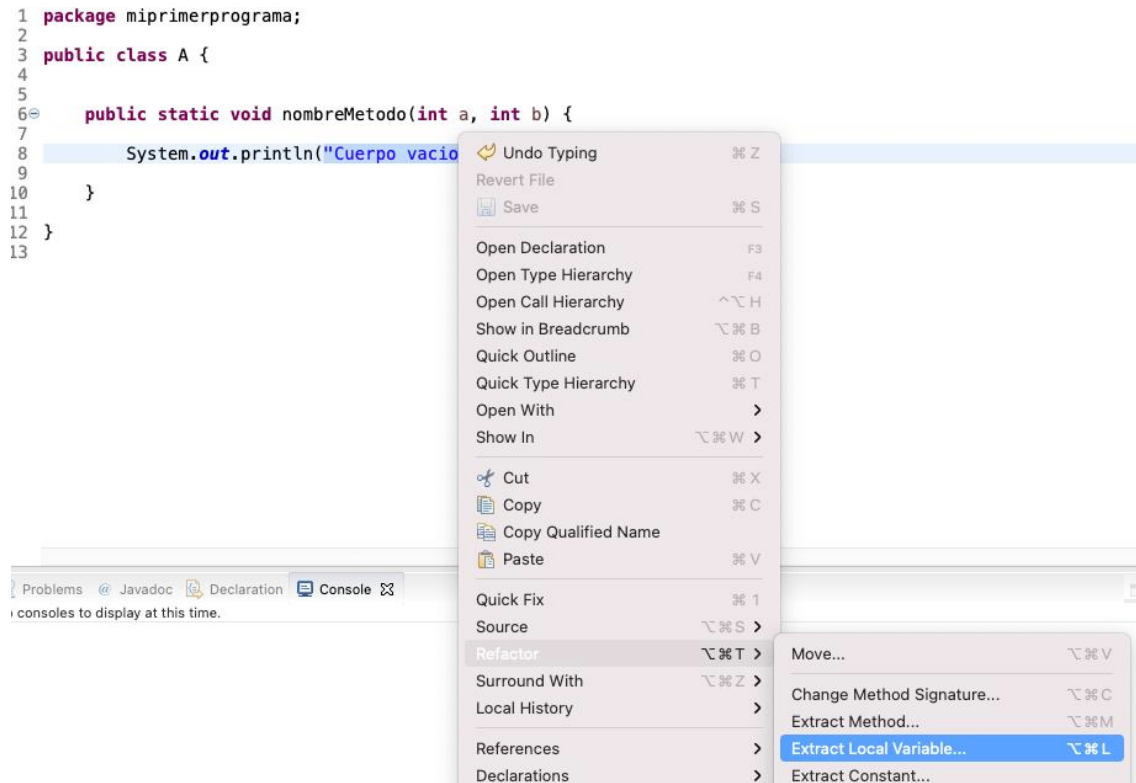
En la siguiente imagen se ve como es posible el acceso a la ventana de dialogo para cambiar la signatura del método *nombreMetodo*.



Extraer variable local.

Crear una variable local inicializada con el valor de un literal (número, String...). Las referencias a esa expresión se modifican por una referencia a la variable.

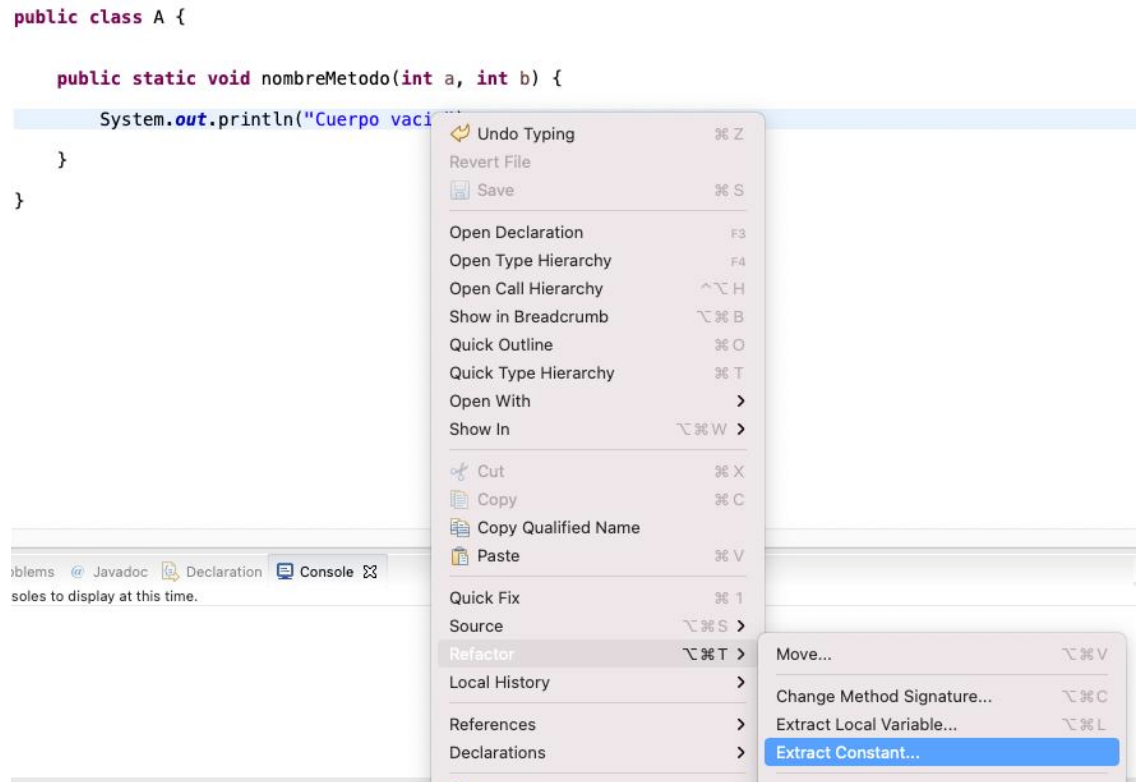
En el ejemplo siguiente se ve como es posible extraer la cadena "Cuerpo vacio" a una variable local del método.



Extraer constante

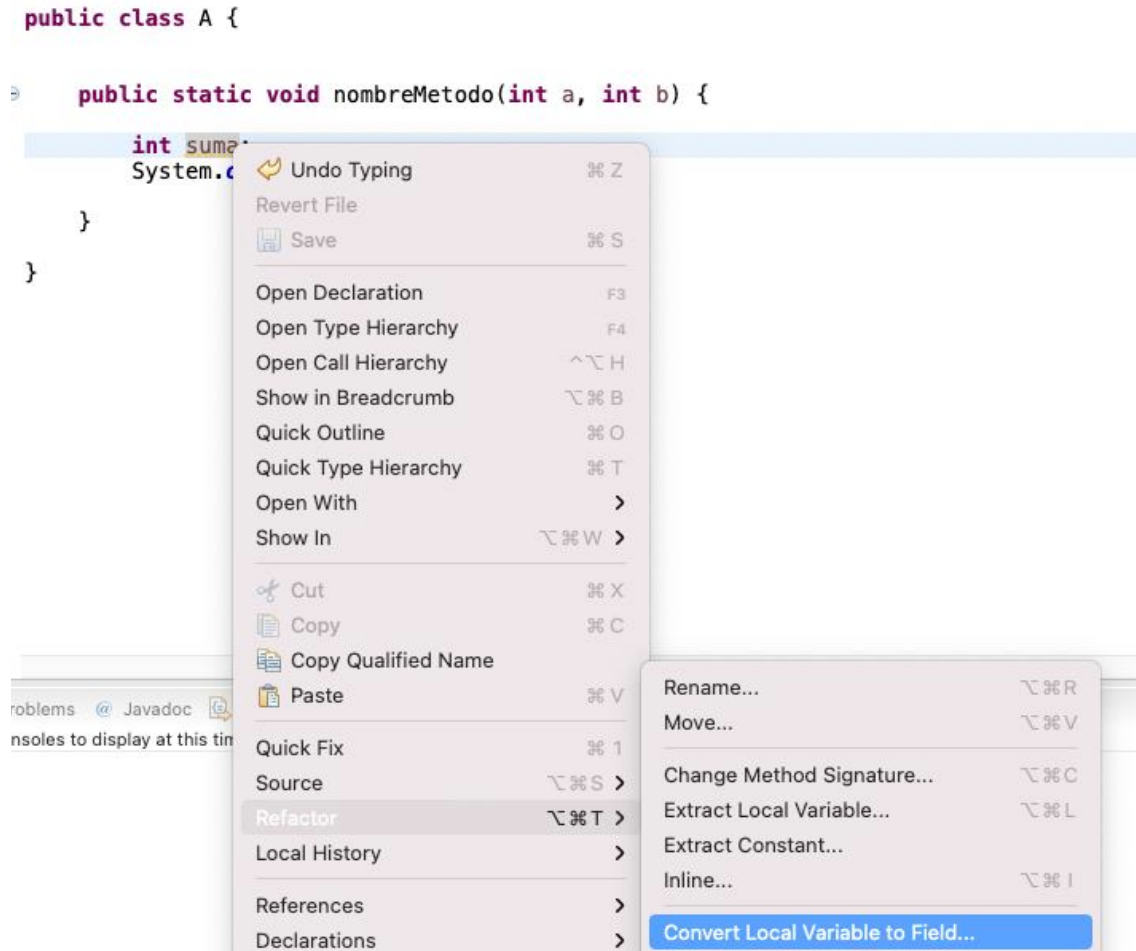
Exactamente igual que el anterior, pero genera una constante con la expresión seleccionada.

En la siguiente imagen veremos como la cadena "Cuerpo vacio" se puede refactorizar convirtiendolo en una constante de clase.



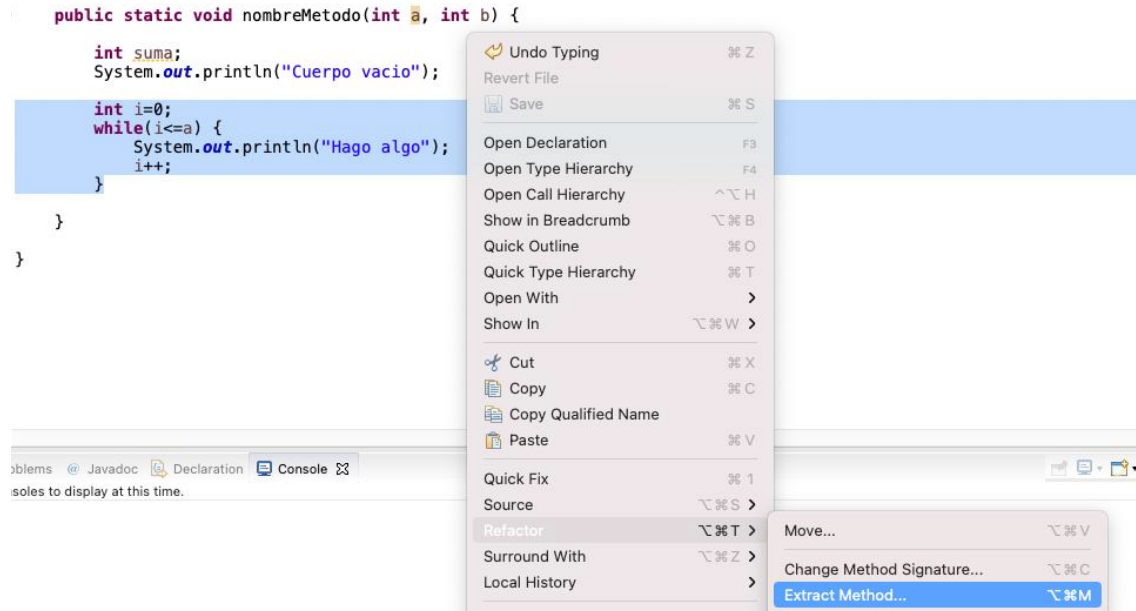
Convertir variable local en atributo.

A veces se definen variables locales dentro de un método que luego resultan ser relevantes en el ámbito de la clase, por tanto debe ser considerada un atributo de la clase.



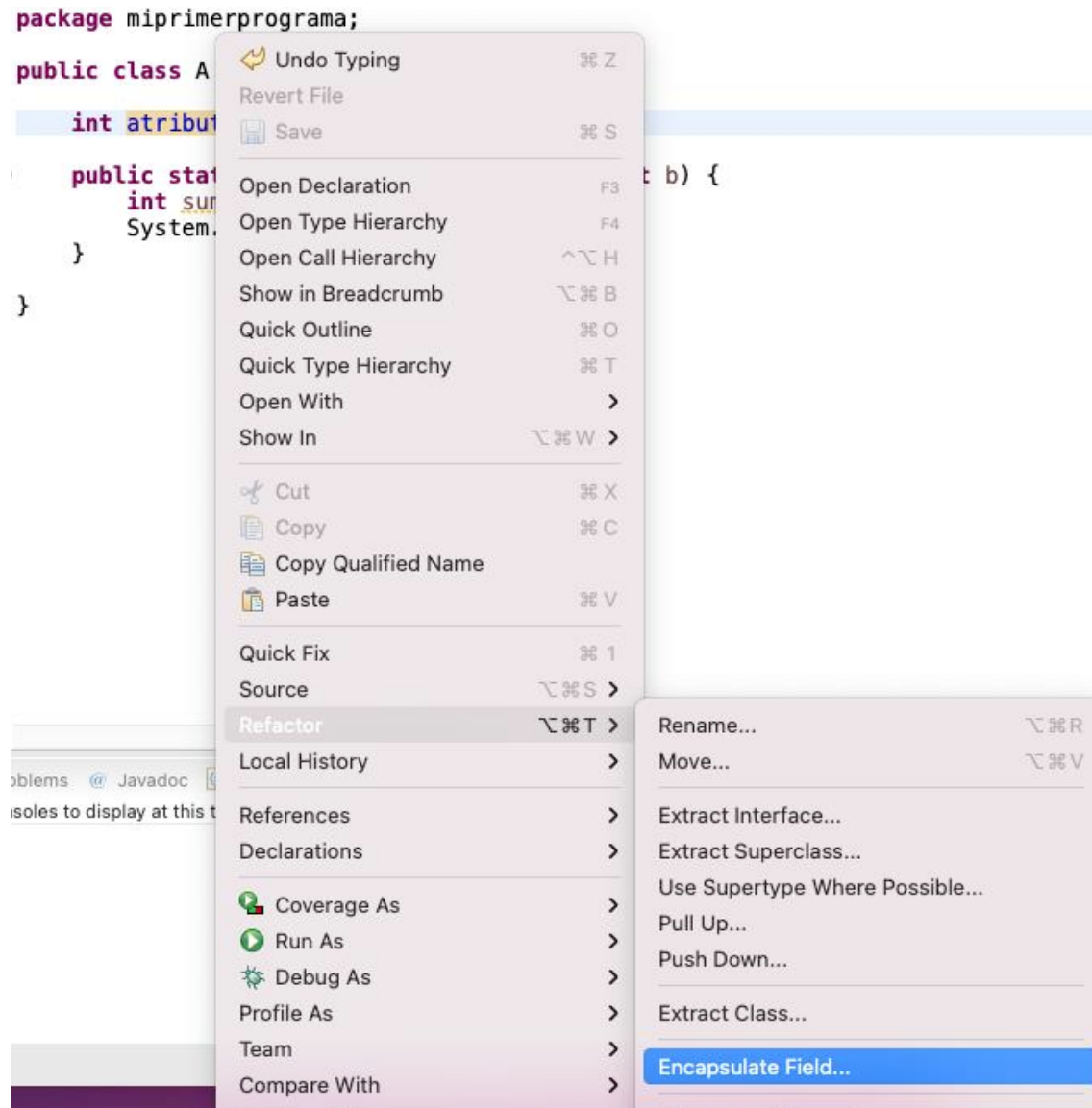
Extraer método.

Convierte el código seleccionado en un método, útil en código que es reutilizado en varios sitios del programa. También puede servir para aligerar un método que es demasiado largo.



Autoencapsular atributo.

Convierte una variable de clase en privada y genera los métodos *Get* y *Set* públicos para acceder a la misma. Opción también disponible desde el menú *código fuente* que se verá a continuación.



Ejercicio

Los pasos a seguir son los siguientes:

Crear el proyecto:

- Crear un proyecto Eclipse llamado "refactor".
- Crear un paquete "utilidades" y dentro crear la clase Circulo.
- Crear un paquete "figuras" y dentro crear la clase Test.

Nota: puedes ver el código más adelante. Pulsa [aquí](#) si quieres descargarlo.

Utilizando las opciones del menú Código fuente, resuelve los siguientes pasos:

- Generar métodos get y set para la clase Circulo.
- Corregir la tabulación del código.
- Dar formato al código.

Utilizando las opciones del menú Refactorizar, resuelve los siguientes pasos:

- Mover la clase Circulo al paquete figuras.
- Renombrar la clase Circulo por Circunferencia. Observar si el cambio afecta a otras clases (en este caso Test).
- Renombrar el atributo "rad" por "radio". ¿Cómo afecta al método get?.
- Convertir la variable local "color" del método imprimir en un atributo, inicializando su valor en el mismo método imprimir.
- En imprimir, en lugar de calcular y escribir el diámetro directamente en el println, extraer a una variable local "d" e imprimir dicha variable.
- Hacer que 3.1416 sea una constante llamada PI.
- Extraer el cálculo del área a un método llamado calcularArea. No recibirá parámetros y devolverá un double.
- Cambiar la firma o cabecera del método esIgual, invirtiendo el orden de los parámetros y cambiando el nombre de conDecimales por considerarDecimales. ¿Cómo afecta el cambio a la clase Test, en la que se usaba este método?.
- Ahora se propone usar "inline" para deshacer algunos cambios, es decir, hacer el código más concreto. Seleccionar la variable "d" (diámetro) y hacer que su valor se use en línea, desapareciendo por tanto la variable.
- Seleccionar la llamada al método calcularArea y hacer que su código se incorpore en la misma línea, desapareciendo la necesidad de usar el método (se puede borrar el método después).
- Seleccionar la constante PI y hacer que su valor se incorpore a las líneas en que se usa, desapareciendo por tanto la constante.
- Se propone repetir el paso anterior con el atributo "color". ¿Es posible eliminar este atributo y utilizar su valor en línea?.

Código de las dos clases:

Clase Circulo (paquete utilidades)

```
package utilidades;
public class Circulo {
    private double rad;
    public Circulo(double radio)
    {
        this.rad = radio;
    }
    public void imprimir()
    {
        String color = "rojo";
        System.out.println("Diámetro: " + 2*rad);
        System.out.println("Color: " + color);
        double area = 2 * 3.1416 * rad * rad;
        System.out.println(area);
    }
    public boolean esIgual (Circulo otro, boolean conDecimales)
    {
        double radio1 = this.rad;
        double radio2 = otro.getRad();
        if (conDecimales)
        {
            if (radio1 == radio2)
                return true;
            else
                return false;
        }
        else
        {
            if (Math.abs(radio1-radio2)<1)
                return true;
            else
                return false;
        }
    }
}
```

Clase Test (paquete figuras)

```
package figuras;
public class Test {
    public static void main (String[] args)
    {
        Circulo c1 = new Circulo(5.5);
        Circulo c2 = new Circulo(10.1);
        Circulo c3 = new Circulo(10.9);

        if (c2.esIgual(c3, false))
        {
            System.out.println("c2 y c3: iguales sin considerar
decimales");
        }

        if (c2.esIgual(c3, true))
        {
            System.out.println("c2 y c3: iguales considerando
decimales");
        }
    }
}
```